

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

Дисциплина интернет-технологии и веб-программирование

ОТЧЁТ
к лабораторной работе
на тему

**АНАЛИЗ СЕТЕВЫХ ПРОТОКОЛОВ И
ИНСТРУМЕНТЫ РАЗРАБОТЧИКА**

Выполнили

В. М. Равгейша
А. А. Почиковский

Проверил

Н. В. Хаджинова

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Приобретение практических навыков анализа сетевого трафика веб-приложений с использованием инструментов разработчика браузера. Изучение структуры *HTTP/HTTPS* запросов и ответов, анализ заголовков, кодов состояния и принципов клиент-серверного взаимодействия.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Клиент-серверная архитектура является основой взаимодействия в сети интернет, где роли распределены между двумя сторонами. Клиент, в роли которого обычно выступает браузер, инициирует запрос к серверу для получения ресурсов, таких как *HTML*-страницы или данные. Сервер постоянно находится в режиме ожидания, обрабатывает поступающие запросы и отправляет соответствующие ответы, обеспечивая централизованное хранение информации.

Модель *OSI (Open Systems Interconnection)* представляет собой многоуровневую архитектуру, состоящую из семи различных уровней, таких как физический, канальный, сетевой, транспортный, прикладной и другие. Каждый уровень выполняет определенные функции, что облегчает разработку и взаимодействие систем.

Модель *TCP/IP (Transmission Control Protocol/Internet Protocol)* – сетевая модель передачи данных, представленных в цифровом виде. Модель описывает способ передачи данных от источника информации к получателю. В модели предполагается прохождение информации через четыре уровня, каждый из которых описывается правилом (протоколом передачи). Наборы правил, решающих задачу по передаче данных, составляют стек протоколов передачи данных, на которых базируется Интернет. Название *TCP/IP* происходит из двух важнейших протоколов семейства – *Transmission Control Protocol (TCP)* и *Internet Protocol (IP)*, которые были первыми разработаны и описаны в данном стандарте.

Уровень приложения является самым верхним в модели *TCP/IP*. Работа *web*-приложений связана с протоколом *HTTP* данного уровня. *HTTP (Hypertext Transfer Protocol)* изначально предназначался для передачи данных в виде *HTML*-документов, а сегодня отвечает за передачу любых данных в клиент-серверном приложении. Программное обеспечение для работы с ним делят на три категории: серверы (обрабатывают запросы), клиенты (отправляют запросы) и прокси (службы-посредники, которые могут кэшировать данные или защищать клиента).

Основным протоколом для передачи веб-страниц в интернете является *HTTP (Hypertext Transfer Protocol)*, который чаще всего использует порты *TCP* 80 или 8080. Он работает по принципу «запрос-ответ», описывая то, как браузеры взаимодействуют с серверами для получения *HTML*-документов, изображений и других ресурсов. Главной особенностью и одновременно недостатком этого протокола является отсутствие безопасности: данные

передаются в открытом текстовом виде, что делает их уязвимыми для перехвата.

Для обеспечения защиты данных используется *HTTPS* (*Hypertext Transfer Protocol Secure*), работающий через порт *TCP* 443. Это защищенная версия протокола, которая использует криптографические алгоритмы *TLS/SSL* для шифрования информации, передаваемой между клиентом и сервером. Благодаря этому обеспечивается высокая конфиденциальность соединений, что делает *HTTPS* обязательным для любых чувствительных операций, таких как банковские транзакции, электронная коммерция и авторизация в личных кабинетах.

Между клиентом и сервером передаются *HTTP*-сообщения, состоящие из стартовой строки (тип запроса и адрес), заголовков (параметры передачи) и тела сообщения (сами данные). Для понимания результата запроса используются коды состояния: 2xx (успех), 3xx (перенаправление), 4xx (ошибка клиента, например, 404), 5xx (ошибка сервера).

Чтобы сервер понял цель обращения, используются методы (запросы):

- *GET* – предназначен для получения данных (параметры передаются прямо в адресе после знака ?).

- *POST* – используется для отправки больших объемов данных (например, форм регистрации), данные передаются в теле сообщения, что более безопасно.

- *HEAD* – аналогичен *GET*, но запрашивает только заголовки (без тела ответа).

- *PUT / PATCH* – для обновления или частичного изменения данных на сервере.

- *DELETE* – для удаления ресурса.

Протокол постоянно развивается: наиболее ходовой сейчас *HTTP/1.1* (с постоянным соединением), *HTTP/2* (уплотнение канала для скорости), а в *HTTP/3* реализован переход на более быстрый транспортный протокол *UDP*.

3 ХОД РАБОТЫ

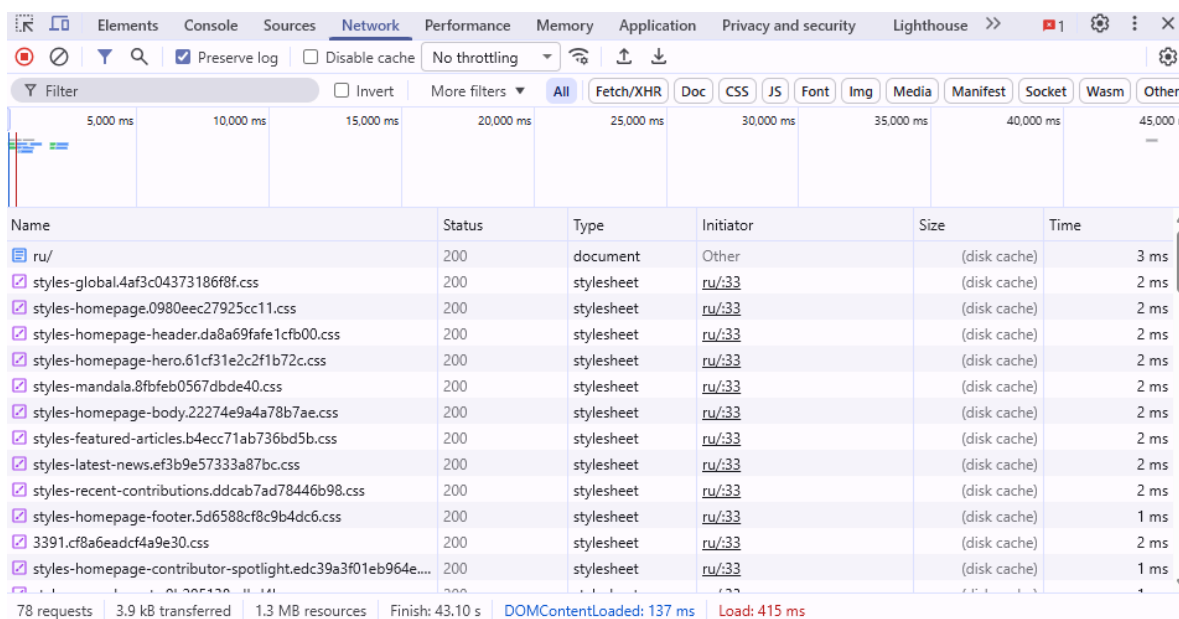
3.1 Подготовка и общее исследование

Для начала работы необходимо запустить браузер *Google Chrome* и вызвать панель инструментов разработчика. Это делается с помощью клавиши «*F12*» или комбинации клавиш «*Ctrl+Shift+I*». В открывшемся окне следует перейти на вкладку *Network* (Сеть), которая предназначена для мониторинга всех сетевых транзакций между браузером и внешними серверами.

Перед фиксацией трафика необходимо убедиться, что инструмент готов к записи. Кнопка «*Record network log*» должна быть активна (подсвечена красным цветом). Также рекомендуется установить галочку «*Preserve log*» («Сохранять журнал») – это критически важно для того, чтобы записи запросов не удалялись при автоматических перенаправлениях или переходе

между страницами. Перед началом замера следует нажать кнопку «*Clear*» («Очистить»), чтобы удалить историю предыдущих соединений.

В адресной строке браузера вводится *URL*-адрес сайта. После нажатия клавиши *Enter* панель *Network* начнет заполняться списком ресурсов. Здесь отображаются все запросы, которые браузер инициирует для полной сборки страницы: от основного *HTML*-документа до вспомогательных скриптов (*JS*), стилей (*CSS*), изображений и *API*-вызовов. Вкладка *Network* в панели разработчика после загрузки сайта показана на рисунке 1.



Name	Status	Type	Initiator	Size	Time
ru/	200	document	Other	(disk cache)	3 ms
styles-global.4af3c04373186f8f.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-homepage.0980eec27925cc11.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-homepage-header.da8a69fafe1cfb00.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-homepage-hero.61cf31e2c2f1b72c.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-mandala.8fbfeb0567dbde40.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-homepage-body.22274e9a4a78b7ae.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-featured-articles.b4ecc71ab736bd5b.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-latest-news.ef3b9e57333a87bc.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-recent-contributions.ddcab7ad78446b98.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-homepage-footer.5d6588cf8c9b4dc6.css	200	stylesheet	ru/:33	(disk cache)	1 ms
3391.cf8a6eadcf4a9e30.css	200	stylesheet	ru/:33	(disk cache)	2 ms
styles-homepage-contributor-spotlight.edc39a3f01eb964e....	200	stylesheet	ru/:33	(disk cache)	1 ms

Рисунок 1 – Вкладка *Network*

На рисунке 1 на вкладке *Network* представлены ключевые столбцы, описывающие процесс загрузки ресурсов. В столбце *Name* отображаются имена запрашиваемых объектов, где первым идет основной документ «*ru/*», а за ним следуют файлы стилей (например, *styles-global.css*). Столбец *Status* указывает код ответа сервера: значение 200 во всех строках подтверждает, что запросы выполнены успешно. В колонке *Type* указаны типы данных – *document* для самой страницы и *stylesheet* для таблиц стилей. *Initiator* показывает источник запроса: для главной страницы это *Other*, а для стилей – «*ru/:33*». Столбец *Size* информирует о размере данных, где пометка (*disk cache*) указывает на загрузку из памяти браузера. В столбце *Time* показывается время выполнения запросов. Самым первым в списке идет запрос к основному ресурсу под именем «*ru/*» с типом *document*. Его статус – 200, что подтверждает успешную загрузку базовой *HTML*-страницы.

3.2 Детальный анализ запросов и ответов

Для проведения детального анализа сетевого взаимодействия необходимо выбрать конкретный запрос из списка и открыть вкладку *Headers*. В этой вкладке в разделе *General* отображается общая информация о запросе:

его полный адрес (*Request URL*), используемый метод (например, *GET*) и итоговый код состояния (*Status Code*). На рисунке 2 показана вкладка *Headers* для запроса загрузки страницы.

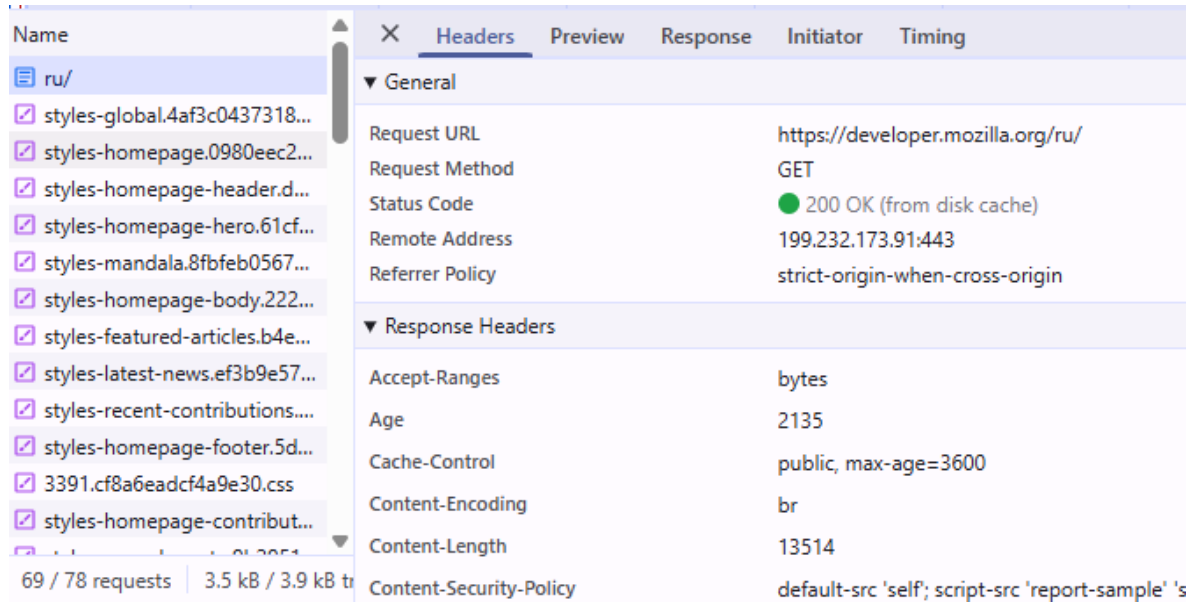


Рисунок 2 – Вкладка *Headers* для запроса загрузки страницы

В таблице 1 подробно описаны пять заголовков ответов для запроса загрузки страницы.

Таблиц 1 – Описание *Response Headers* для запроса загрузки страницы

Заголовок	Значение	Назначение
<i>Cache-Control</i>	<i>public, max-age=3600</i>	Управляет правилами кэширования; разрешает хранение ресурса в кэше браузера или прокси-сервера на 3600 секунд (1 час)
<i>Content-Length</i>	13514	Указывает точный размер тела ответа (передаваемых данных) в байтах.
<i>Content-Type</i>	<i>text/html</i>	Определяет тип медиаданных ресурса; значение <i>text/html</i> сообщает браузеру, что передается <i>HTML</i> -документ.
<i>Accept-Ranges</i>	<i>bytes</i>	Указывает, что сервер поддерживает частичную загрузку данных по байтам (позволяет возобновлять прерванную загрузку)
<i>date</i>	<i>Tue, 27 Jan 2026 22:21:07 GMT</i>	Содержит дату и точное время, когда сервер сформировал и отправил данный ответ.

В таблице 2 подробно описаны пять заголовков запросов для запроса загрузки страницы.

Таблиц 2 – Описание *Request Headers* для запроса загрузки страницы

Заголовок	Значение	Назначение
<i>sec-ch-ua</i>	"Not(A:Brand";v="8", "Chromium";v="144", "Google Chrome";v="144"	Предоставляет серверу краткую и структурированную информацию о названии и версии браузера (современная замена или дополнение к <i>User-Agent</i>).
<i>sec-ch-ua-mobile</i>	?1	Указывает серверу, был ли сделан запрос с мобильного устройства (значение ?1 означает «да»).
<i>sec-ch-ua-platform</i>	"Android"	Сообщает серверу название операционной системы пользователя (в данном примере – <i>Android</i>).
<i>upgrade-insecure-requests</i>	1	Сообщает серверу, что браузер поддерживает автоматический переход с <i>HTTP</i> на более защищенный <i>HTTPS</i> .
<i>user-agent</i>	Mozilla/5.0 (Linux; Android 6.0; Nexus 5 Build/MRA58N) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Mobile Safari/537.36	Содержит полную строку идентификации клиента: название и версию браузера, операционную систему, тип устройства и используемый движок отображения страниц.

На рисунке 3 показана вкладка *Headers* для запроса загрузки картинки.

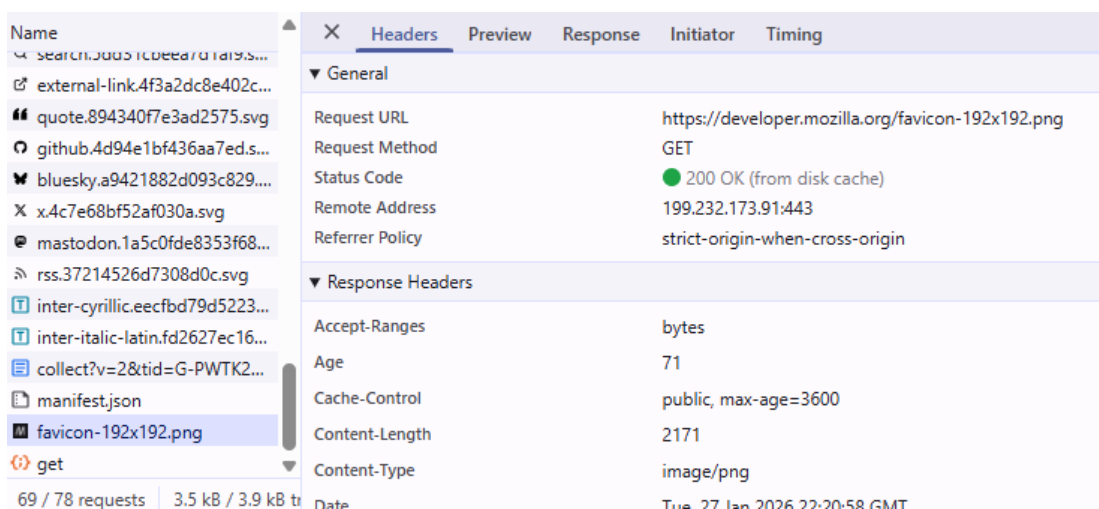


Рисунок 3 – Вкладка *Headers* для запроса загрузки изображения

В разделе *Response Headers* анализируются заголовки, которые сервер прислал браузеру для управления полученными данными. Раздел *Request Headers* содержит заголовки, отправленные браузером серверу для уточнения параметров запроса.

Дополнительно для анализа могут использоваться вкладки *Preview* и *Response*, которые позволяют обработанный ответ от сервера (например, отрендеренную *HTML*-страницу или *JSON*-данные). На рисунке 4 показана вкладка *Preview*.

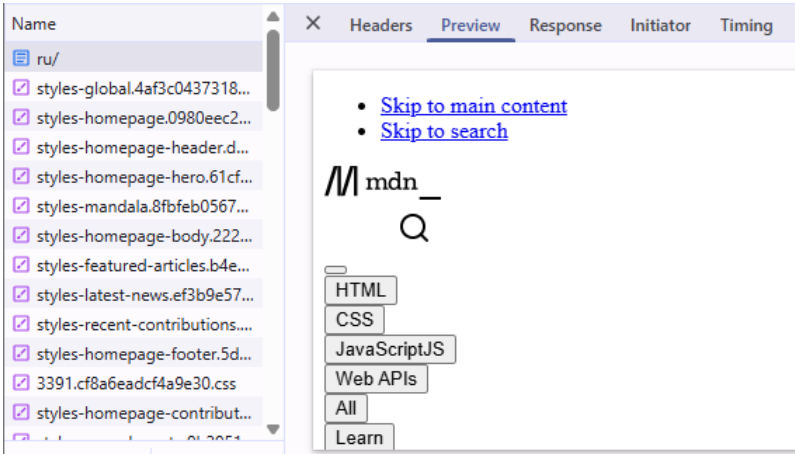


Рисунок 4 – Вкладка *Preview*

Вкладка *Timing* показывает детальную временную диаграмму выполнения запроса (*DNS*-поиск, установка соединения, *SSL*-рукопожатие, ожидание ответа сервера и т.д.). На рисунке

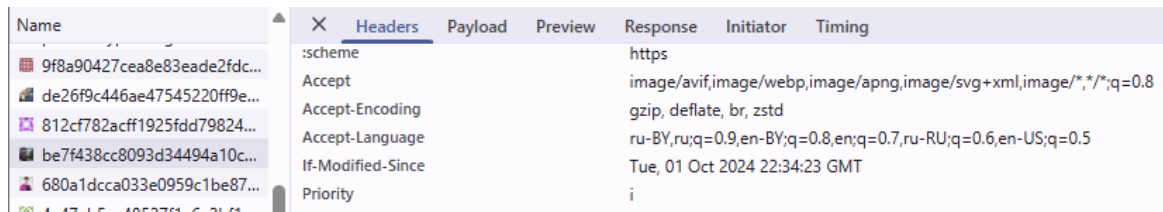
3.3 Выполнение индивидуального задания

Откроем сайт <https://stackoverflow.com/> и найдем запросы, которые возвращают статус *304 Not Modified*. Этот статус – информирует о том, что ресурс не был изменен с конкретного момента, будет возвращен только если исходный запрос был произведен методом *GET* с содержанием заголовка *If-Modified-Since* или *If-None-Match*. Используется для кэширования, чтобы клиент не загружал данные повторно. На рисунке 5 представлены несколько запросов со статусом *304*.

Name	Status	Type	Initiator	Size	Time
9f8a90427cea8e83eade2fdc10dab32a?s=32&d=identicon...	304	png	questions:2837	0.3 kB	290 ms
de26f9c446ae47545220f9eff3e3ef7?s=32&d=identicon&r...	304	jpeg	questions:2837	0.4 kB	291 ms
812cf782acff1925fdd79824538994bf?s=32&d=identicon&r...	304	png	questions:2837	0.3 kB	291 ms
be7f438cc8093d34494a10cd99562313?s=32&d=identicon...	304	png	questions:2837	0.4 kB	291 ms
680a1dcca033e0959c1be87a1800ae60?s=32&d=identicon...	304	png	questions:2837	0.4 kB	291 ms
4e47eb5ec48527f1e6a2bf14c2032b79?s=32&d=identicon...	304	png	questions:2837	0.3 kB	291 ms
4277330c80daa1e64a68760000b387f4?s=32&d=identicon...	304	png	questions:2837	0.3 kB	291 ms
f13ec04013f3ac623d3390a4daac425?s=32&d=identicon&...	304	png	questions:2837	0.3 kB	291 ms
058dc9db58942c2f6503ba0517b6195f?s=32&d=identicon...	304	png	questions:2837	0.3 kB	289 ms
0623971232577218e4a86bcd7753f00?s=32&d=identicon...	304	png	questions:2837	0.3 kB	289 ms

Рисунок 5 – Запросы со статусом *304 Not Modified*

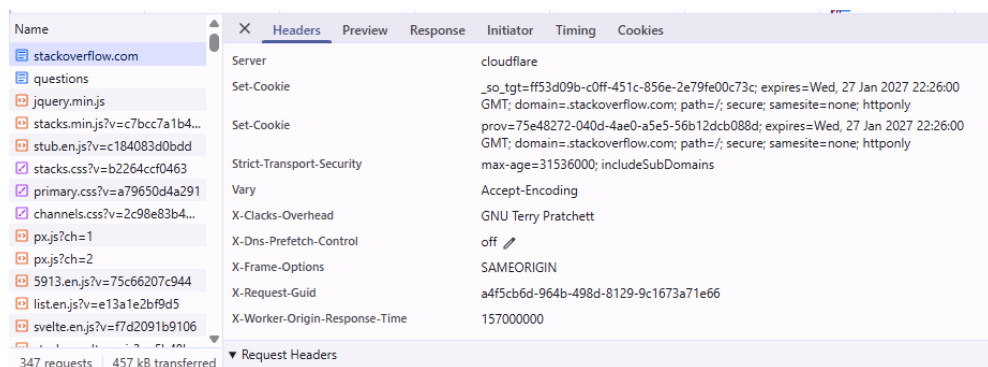
Проверим один из запросов на наличие заголовка *If-Modified-Since* или *If-None-Match*. Результат показан на рисунке 6.



Name	Value
:scheme	https
Accept	image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	ru-BY,ru;q=0.9,en-BY;q=0.8,en;q=0.7,ru-RU;q=0.6,en-US;q=0.5
If-Modified-Since	Tue, 01 Oct 2024 22:34:23 GMT
Priority	i

Рисунок 6 – Проверка заголовка *If-Modified-Since*

Теперь разберем заголовок *Set-Cookie*. На рисунке 6 представлены эти заголовки для запроса на загрузку страницы.

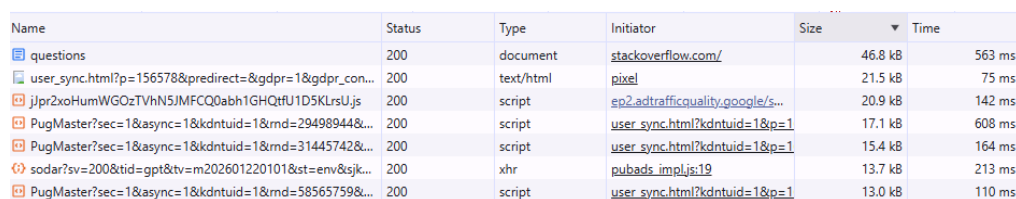


Name	Value
Server	cloudflare
Set-Cookie	_so_tgt=ff53d09b-c0ff-451c-856e-2e79fe00c73c; expires=Wed, 27 Jan 2027 22:26:00 GMT; domain=.stackoverflow.com; path=/; secure; samesite=none; httponly
Set-Cookie	prov=75e48272-040d-4ae0-a5e5-56b12dcb088d; expires=Wed, 27 Jan 2027 22:26:00 GMT; domain=.stackoverflow.com; path=/; secure; samesite=none; httponly
Strict-Transport-Security	max-age=31536000; includeSubDomains
Vary	Accept-Encoding
X-Clacks-Overhead	GNU Terry Pratchett
X-Dns-Prefetch-Control	off
X-Frame-Options	SAMEORIGIN
X-Request-Guid	a4f5cb6d-964b-498d-8129-9c1673a71e66
X-Worker-Origin-Response-Time	157000000

Рисунок 7 – Заголовки *Set-Cookie*

По параметру *expires* видно, что данные *cookie* не являются сессионными, так как имеют длительный срок жизни в год, что нехарактерно для сессий и в то же время избыточно для хранения пользовательских предпочтений. Наличие флага *HttpOnly*, запрещающего доступ к *cookie* со стороны *JavaScript*, указывает на то, что они не используются для *UI*-настроек или клиентских предпочтений. Флаг *Secure*, означающий передачу *cookie* только по *HTTPS*, говорит о том, что они применяются в контексте безопасного обмена данными. Следовательно, можно сделать вывод, что эти *cookie* предназначены для аутентификации пользователя или защиты и управлением трафиком и корректной работы сайта.

Теперь отсортируем запросы по столбцу *Size* и посмотрим какие имеют наибольший размер. Отсортированный список запросов показан на рисунке 8.



Name	Status	Type	Initiator	Size	Time
questions	200	document	stackoverflow.com/	46.8 kB	563 ms
user_sync.html?p=156578&predirect=&gdpr=1&gdpr_con...	200	text/html	pixel	21.5 kB	75 ms
jpr2xoHumWGOzTVhN5JMFQ0abh1GHQttU1D5KLsUjs	200	script	ep2.adtrafficquality.google/s...	20.9 kB	142 ms
PugMaster?sec=1&asyn=1&kdntuid=1&rnd=29498944&...	200	script	user_sync.html?kdntuid=1&p=1	17.1 kB	608 ms
PugMaster?sec=1&asyn=1&kdntuid=1&rnd=31445742&...	200	script	user_sync.html?kdntuid=1&p=1	15.4 kB	164 ms
sodar?sv=200&tid=gpt&tv=m202601220101&st=env&sjk...	200	xhr	pubads_impl.js:19	13.7 kB	213 ms
PugMaster?sec=1&asyn=1&kdntuid=1&rnd=58565759&...	200	script	user_sync.html?kdntuid=1&p=1	13.0 kB	110 ms

Рисунок 8 – Список самых больших запросов

На рисунке 8 мы видим исключение из общего правила: здесь самыми «тяжелыми» оказались *document* (46.8 кБ) и *script* (21 кБ). Это объясняется тем, что на этом сайте отсутствуют изображения только маленькие иконки. Наибольший размер в *HTTP*-запросах обычно имеют изображения, так как они содержат бинарные данные высокой плотности, часто занимающие сотни килобайт или мегабайты. На большинстве современных сайтов (интернет-магазины, новостные порталы, соцсети) именно картинки (форматы *jpeg*, *png*, *webp*) занимают от 60% до 80% общего веса страницы, качественное фото может весить несколько мегабайт, что в десятки раз больше любого скрипта. И правда, если проанализировать другие сайты, то так оно и есть. *JavaScript*-файлы чаще всего занимают второе место, а *CSS* – самые лёгкие среди этих типов ресурсов.

4 ВЫВОД ПО РАБОТЕ

В ходе выполнения лабораторной работы были приобретены практические навыки анализа сетевого трафика веб-приложений с использованием инструментов разработчика браузера (вкладка *Network*). На практике было изучено взаимодействие клиента и сервера на прикладном уровне моделей *OSI* и *TCP/IP*.

В процессе исследования были проанализированы структуры *HTTP*-запросов и ответов. На основе детального разбора разделов *Request Headers* и *Response Headers* была установлена роль ключевых заголовков (*Content-Type*, *Cache-Control*, *User-Agent* и др.) в управлении сессиями, кэшировании и корректным отображением контента.

В рамках выполнения индивидуального задания был проведен анализ *Cookie*. Изучение атрибутов *HttpOnly*, *Secure* и сроков жизни (*Expires*) позволило сделать вывод об использовании данных файлов для долгосрочной аутентификации и защиты трафика. Анализ распределения ресурсов по их объему показал, что вес страницы напрямую зависит от специфики контента: на текстовых ресурсах основной объем трафика приходится на *JavaScript*-сценарии, в то время как на большинстве современных веб-порталов лидирующим типом данных являются изображения.

Таким образом, цель работы полностью достигнута. Полученные навыки мониторинга сетевых протоколов являются базовыми для диагностики работы веб-приложений, анализа их производительности и обеспечения безопасности передачи данных.